

S&Box

Module Documentation



Prepared By S&BOX
(Group 12)

1.Group members

- Ahmet Eren ARSLAN
- Mert UĞUR
- Bekir Sadık ALTUNKAYA
- Musab KARDEŞ
- Beyzanur YILMAZ
- Mehmet Emin BAYTEKİN
- Ahmet Hakan DEMİR
- Muhammet Talha MEMİŞOĞLU

2. Introduction

Game Name: S&BOX

Project Scope: This project aims to scan a physical surface containing sand and transform it into a playable game map. The variations in sand height will be analyzed to create a dynamic game environment that players can explore. If the shape of the terrain is altered, the game map will automatically update in real time to reflect the changes.

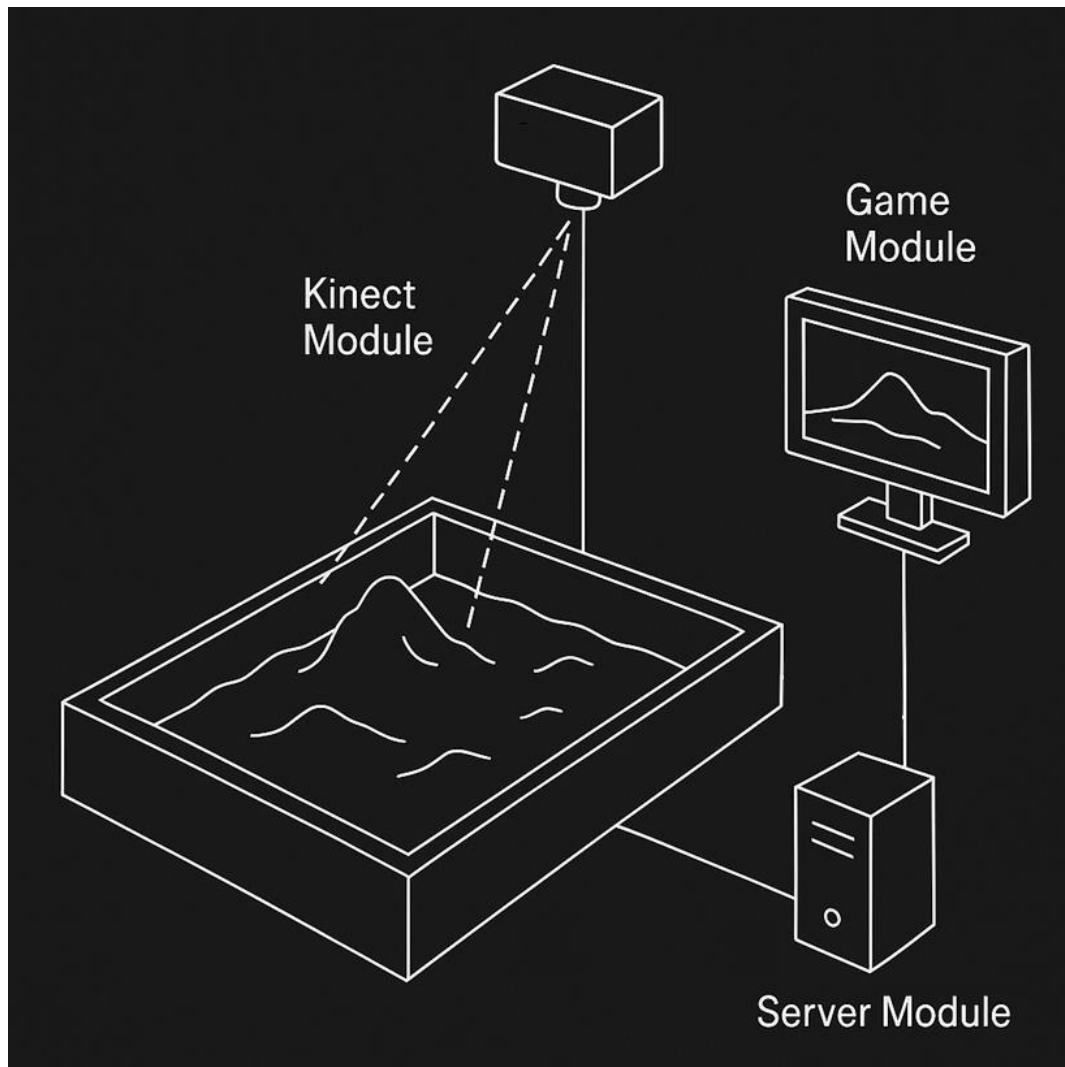
The Aim of The Project: The main goal of the project is to bridge the physical and digital worlds, providing players with a unique and interactive gaming experience. Players will be able to enjoy the game in a multiplayer mode.

3. Modules

Modules are divided to manage the project efficiently and distribute responsibilities across the team. The project consists of two main parts: game development and tool development. We divided it into 3 modules as follows:

- **Game Module:**
Covers the full development of the game, including core mechanics, real-time terrain updates based on Kinect data, multiplayer synchronization, and the user interface. It ensures smooth interaction and visual representation of the dynamic environment.
- **Kinect Module:**
Captures and processes depth data from the sand surface using Kinect, filters noise, and transfers the data to the game in real-time.
- **Server and Streaming Module:**
Hosts the multiplayer game via Unreal Engine Dedicated Server and streams it to web/mobile platforms using Pixel Streaming through Vagon.io.

Each module is designed to work together seamlessly, providing a real-time and interactive gaming experience across devices.



Kinect Module

Module Description:

This module constitutes the fundamental data collection and surface analysis component of the S&BOX project. It encompasses the processes of collecting depth data from physical surfaces using the Xbox Kinect One sensor, processing this data, and transferring it to the game environment in real-time. The module creates a bridge between the physical world and the digital game, ensuring that changes made by users on the sand surface are instantly reflected on the game map.

Requirements:

- **Data Collection and Processing:**
 - Continuous collection of depth data using the Xbox Kinect One sensor
 - Filtering of collected raw depth data and noise removal
 - Detection of height differences on the sand surface
 - Real-time data transfer to the server side when changes are detected
- **Calibration and Configuration:**
 - Positioning of the Kinect sensor at optimal distance and angle
 - Determination and limitation of the scanning area
 - Calibration settings for proper scaling of depth data
- **Integration:**
 - Configuration of the Kinect according to project requirements
 - Integration of processed data with the server module
 - Determination of protocol and format for real-time data transfer

Technologies and Tools:

- **Xbox Kinect One:** *Collection of depth image data*
- **C++:** Programming language used to process Kinect data
- **Data Processing Algorithms:** Depth map creation, filtering, and difference detection

Data Processing Flow:

1. **Data Collection:** Continuous scanning of the sand surface using Xbox Kinect One and obtaining depth data
2. **Pre-processing:** Filtering, noise removal, and normalization of raw depth data
3. **Difference Analysis:** Detection of significant changes between current and previous scans
4. **Data Transformation:** Conversion of depth data into a format usable by the game engine
5. **Data Transfer:** Real-time transfer of processed data to the server module

Testing and Debugging:

- Test protocols for Kinect sensor calibration and accuracy
- Evaluation of sensor performance under different lighting conditions
- Data transmission speed and latency tests
- Measurement of accuracy rate in detecting surface changes

Timeline and Milestones:

1. Setup of Kinect sensor and configuration of basic data collection operations
2. Development of depth data filtering and processing algorithms
3. Implementation and optimization of the change detection system
4. Completion of data integration with the server module
5. Comprehensive testing and performance optimization

Design and Architecture:

• Architecture:

- **Data Acquisition:** *Raw depth data capture from Kinect sensor.*
- **Pre-processing:** Noise filtering and normalization.
- **Change Detection:** Delta analysis between consecutive scans.
- **Data Transformation:** Conversion to game-compatible format (2D array).
- **Integration:** Real-time data transfer to the server module.

• Design Patterns:

- **Observer Pattern:** *Kinect data listeners trigger processing when new scans arrive.*
- **Pipeline Pattern:** Sequential stages (capture -> filter -> detect -> transform -> transmit).
- **Singleton Pattern:** Centralized access to Kinect hardware resources.

Core Components:

1. Kinect Sensor Interface:

- Handles low-level communication with Xbox Kinect.
- Capture raw depth frames at configured resolution.
- 2. Data Processor:
 - Filtering and removing noise.
 - Normalize depth values to a consistent range.
- 3. Change Detector:
 - Compares current frame with previous scan.
 - Flags regions with significant changes.
- 4. Array Converter:
 - Flattens processed depth data into a 2D array.
 - Maps Kinect coordinates to game world.
- 5. Network Adapter:
 - Serializes the 2D array into a compact binary format for server transmissions.

Test Cases:

Feature	Purpose	Test Input/Action	Expected Result	Result
Connecting Kinect to pc	Checking if Kinect can give us result	Try to connect Kinect and see a result	Kinect gives us a video or 3D data	Done
Kinect Data Retrieval	Verify C++ code can capture Kinect data	Testing a C++ code that takes Kinect data in array	C++ code can capture data and convert it to an array	Done
Kinect Data Validation	Ensure the accuracy of Kinect-sourced data	Check the Kinect output data if it is valid	Output of Kinect data is valid with real data	Done
Data Change Detection	Ensure if the data is change and valid	Check the Kinect data if the data change when the Kinect camera position change	Output data update when Kinect camera position change	Done
Kinect Data Processing	Validate transformation of raw Kinect	Process raw Kinect data (filter it or convert it)	Converting data to a 2D array or another data	Planned

	data for game compatibility		usable for game	
--	-----------------------------	--	-----------------	--

Game Backend and Mechanics

Module Description:

This module covers the development of the game's core backend systems. It primarily focuses on processing and updating the dynamic 3D terrain captured through Kinect scans and gameplay. Additionally, it manages real-time terrain updates, player interactions, movement logic over changing surfaces, and collision detection. Multiplayer synchronization is also a critical aspect to ensure consistent terrain and gameplay states across all clients.

Requirements:

- **Terrain Integration:**
 - Convert Kinect-generated depth data into an in-game heightmap or mesh structure.
 - Continuously update the terrain in real-time as the sand formation changes.
- **Player Movement and Interaction:**
 - Develop movement logic that allows characters to navigate over dynamically changing terrain.
 - Define player interactions including physics-based behaviors and ability usage.
- **Gameplay Implementation:**
 - Design and implement FPS gameplay mechanics integrated with dynamic terrain. Develop core shooting mechanics such as aiming, firing, reloading and hit detection, all adapted to constantly changing terrain. Integrate verticality and terrain shape into combat—allowing players to use deformable surfaces for cover, elevation advantage, or traversal.
 - Design free for all deathmatch game mode and suitable kill counter for all players.
- **Multiplayer Handling:**
 - Implement a networked game architecture to ensure synchronized terrain and gameplay across all clients.
 - Use server-authoritative logic to process Kinect data and distribute terrain updates to all players.
 - Manage player states, inputs, and interactions in a way that maintains consistency.
- **Deployment:**
 - Deploy the game for mobile and PC devices.
- **Testing and Debugging:**

- Conduct unit tests for terrain parsing, player movement, multiplayer synchronization, and core gameplay functions.
- Implement logging systems to track and debug issues related to terrain updates and real-time network events.

Timeline and Milestones:

- Terrain data parsing.
- Player movement system over dynamic terrain.
- Multiplayer architecture setup and synchronization testing.
- Development and refinement of game mechanics and rules.
- Deploy the game.
- Comprehensive testing, debugging, and final optimization.

Design and Architecture

- **Architecture:** Modular layered design with separation for:
 - Terrain system
 - Movement logic
 - Networking logic
 - Core gameplay systems
 - Deployment
- **Design Patterns:**
 - **Observer Pattern:** To react to real-time Kinect changes.
 - **Command Pattern:** For handling player actions consistently in a networked environment.
 - **Singleton Pattern:** To manage shared states like terrain data on the server.
- **Core Components:**
 - **Terrain Manager:** Updates terrain mesh.
 - **Movement Engine:** Governs character navigation over changing surfaces.
 - **Gameplay Controller:** Manages the core game rules, dynamic objectives, and player scoring in the FPS environment. Tracks real-time events like kills.
 - **Multiplayer Manager:** Synchronizes terrain and gameplay across players.
 - **Deployment:** Deploy the game for PC and mobile devices.

Implementation Details

Technologies & Tools:

- **Unreal Engine 5** : Main development platform.
- **Git**: For version control.
- **Visual Studio**: For C++ development and build configuration.

Key Implementations:

- **Convert Kinect data into real-time, deformable terrain meshes**
Process incoming Kinect height map and update the game world's terrain using Unreal Engine's dynamic mesh system, allowing the environment to change during gameplay.
- **Implement collision-aware FPS movement over dynamic terrain**
Develop responsive first-person movement that adapts to terrain deformation, including slope handling, jumping, crouching, and sliding mechanics on uneven surfaces.
- **Use server-authoritative networking for FPS actions and terrain sync**
Leverage Unreal Engine's networking framework or a custom RPC protocol to ensure all combat actions (shooting, aiming, ability use) and terrain changes are processed and replicated consistently across clients.
- **Design a scoring and match progression system for FPS gameplay**
Implement real-time scoring based on player kills and deathmatch game mode.
- **Deploy the game PC and Mobile Devices**
Ensure game itself can be playable in mobile and PC devices

Dependencies and Integration

- **External Dependencies:**
 - **Unreal Engine**: Core platform for rendering and physics.
- **Integration with Other Modules:**
 - **Kinect Module**: Sync terrain map based on data that is collected by Kinect.
 - **Server Module** : Sends/receives terrain and gameplay data.

Test Cases:

Feature	Purpose	Test Input / Action	Expected Result	Result
Terrain Parsing Unit Test	Verify terrain data is parsed correctly for game environment	Run unit tests with various terrain files and check parsed output	Parsed terrain matches input data with no errors	Planned
Flat Terrain Movement Test	Validate basic movement input (WASD or joystick) on flat surface	Simulate walking in all directions on a flat, undeformed terrain	Player moves smoothly without jitter or unintended collisions	Planned

Sloped Terrain Navigation Test	Ensure players can ascend and descend inclined terrain correctly	Place player at the base of an elevated slope and simulate upward/downward movement	Player moves naturally up/down slopes without clipping or sliding unexpectedly	Planned
Jump and Land Behavior Test	Verify proper handling of jumping on varied terrain heights	Simulate a jump across a flat surface and from a hilltop to a lower platform	Player jumps with appropriate height and lands with correct physics reaction	Planned
Edge Detection and Falling Test	Ensure character detects cliff edges and triggers falling appropriately	Move player to the edge of a steep terrain drop-off	Player can fall if walking off edge; falling physics behaves realistically	Planned
Terrain Deformation Mid-Movement Test	Test how player responds to live terrain changes underfoot	Begin walking, then deform terrain (e.g., simulate lowering ground via Kinect input) while moving	Player repositions or falls smoothly without getting stuck or desynced with terrain	Planned
Collision Accuracy Test	Validate movement boundaries and collision shape updates with terrain	Try to walk into or along steep terrain walls or protrusions	Player collides with updated mesh properly and cannot pass through terrain	Planned
Multiplayer Sync Test	Confirm real-time synchronization between players	Simulate multiple clients performing actions simultaneously	All clients reflect consistent and synchronized game state	Planned
Kill Scoring Test	Verify scoring logic for eliminating other players	Simulate a successful hit resulting in a kill (e.g., headshot, body shot)	Player's score increases correctly, kill feed updates, and stats reflect kill	Planned
Respawn Logic Test	Ensure correct respawn behavior and invincibility window	Kill a player and monitor respawn sequence	Player respawns at valid point with short invincibility and correct HUD update	Planned
Ammo and Reload System Test	Validate weapon ammo depletion, reloading, and UI updates	Simulate continuous firing until empty, then reload	Ammo count decreases, reload animation plays, weapon resets properly	Planned
Scoreboard Update Test	Ensure real-time scoreboard reflects all gameplay actions	Perform kills	Scoreboard updates live for all players with correct rankings and stats	Planned

Deployment Test in PC	Ensure game is playable in PCs	Play the game for a while	Everything is smooth and playable in PC	Planned
Deployment Test in Mobile Devices	Ensure game is playable in Mobile Devices	Play the game for a while	Everything is smooth and playable in mobile devices	Planned
Logging System	Track terrain and network event issues during runtime	Initiate terrain changes and multiplayer events with logging enabled	Logs show timestamps, event details, and any errors for debugging	Planned

Unreal Engine Multiplayer Server Module Report

1. Overview

This document details the design, development, and testing of the Server Module for our Unreal Engine multiplayer project. The Server Module is responsible for hosting multiplayer sessions, maintaining synchronization among clients, and ensuring low-latency interaction between players. Our primary goal was to create a smooth and responsive multiplayer experience across various platforms.

The project uses Unreal Engine's Dedicated Server architecture, allowing us to separate game logic from clients, improving authority control and reducing cheating possibilities. In earlier phases, we experimented with Pixel Streaming, a technology that streams gameplay from a GPU-powered cloud server to a client browser using WebRTC. Although functional, Pixel Streaming was found to be resource-intensive and unsuitable for our goals due to high hardware and bandwidth demands.

After evaluating modern mobile hardware and optimizing our game assets, we shifted focus toward native deployment for PC, Android, and iOS platforms instead of using pixel streaming. This decision enabled us to reduce data consumption, improve responsiveness, and enhance overall device compatibility.

2. Requirement Specification

2.1 Functional Requirements

Requirement	Description
Dedicated Server Setup	A server instance must be deployed to manage multiplayer sessions.
Multiplayer Connectivity	Clients must be able to connect, join sessions, leave sessions, and reconnect smoothly.
Session Auto-Join	Clients should automatically connect to the last known server on startup for ease of use.
Platform Support	The game must run on PC, Android, and (if possible) iOS platforms.
Mobile Support	Packaged mobile versions must retain full multiplayer connectivity.

2.2 Non-Functional Requirements

Requirement	Description
Low Latency	Multiplayer sessions must operate with minimal latency for real-time interaction.
Scalability	The server must support multiple players without significant performance loss.
Fault Tolerance	Disconnections and reconnections must be handled gracefully.

Optimization	Network and graphical performance must be acceptable on mobile devices.
Testing on Devices	Real-device testing must validate both network logic and in-game rendering (in progress).

3. Technical Architecture & Analysis

3.1 Tools & Technologies

Component	Purpose
Unreal Engine 4.27	Game engine used for both the client and dedicated server builds. And deployment for mobile and computer platforms
Dedicated Server	Manages authority-based gameplay and client replication.
Git	Version control system for collaborative development.
Node.js	Used for Pixel Streaming signalling server setup.

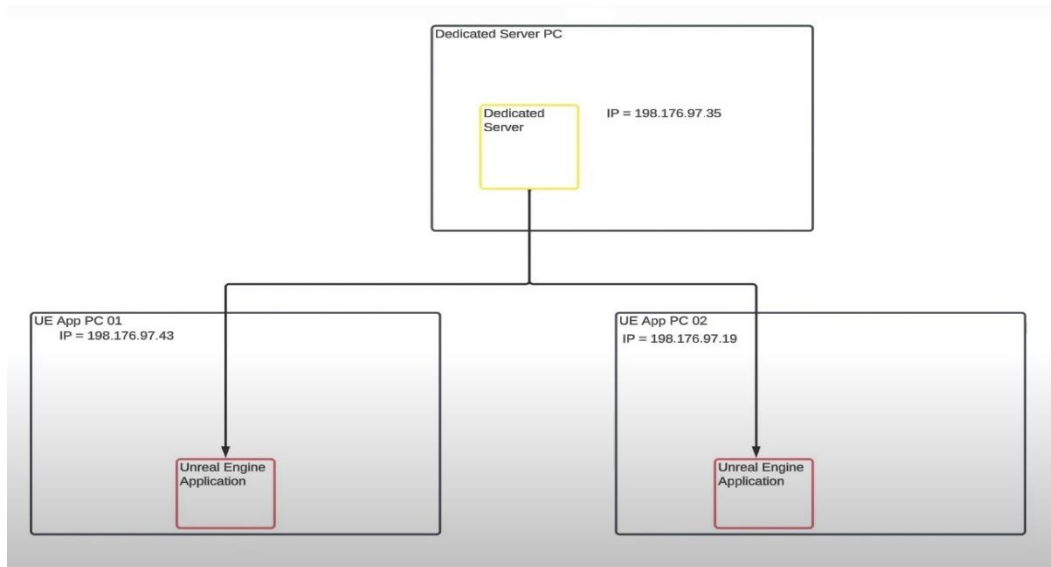
3.2 Dedicated Server Integration

We configured and built a dedicated server using Unreal's UnrealBuildTool. The server was launched with the listen parameter to enable incoming client connections. Testing was performed primarily in local and LAN-based environments to simulate real-world network conditions.

Key features:

- Auto-connect logic implemented to automatically connect the client to the last known IP on startup.
- Session management fully implemented: clients can join, leave, and reconnect.

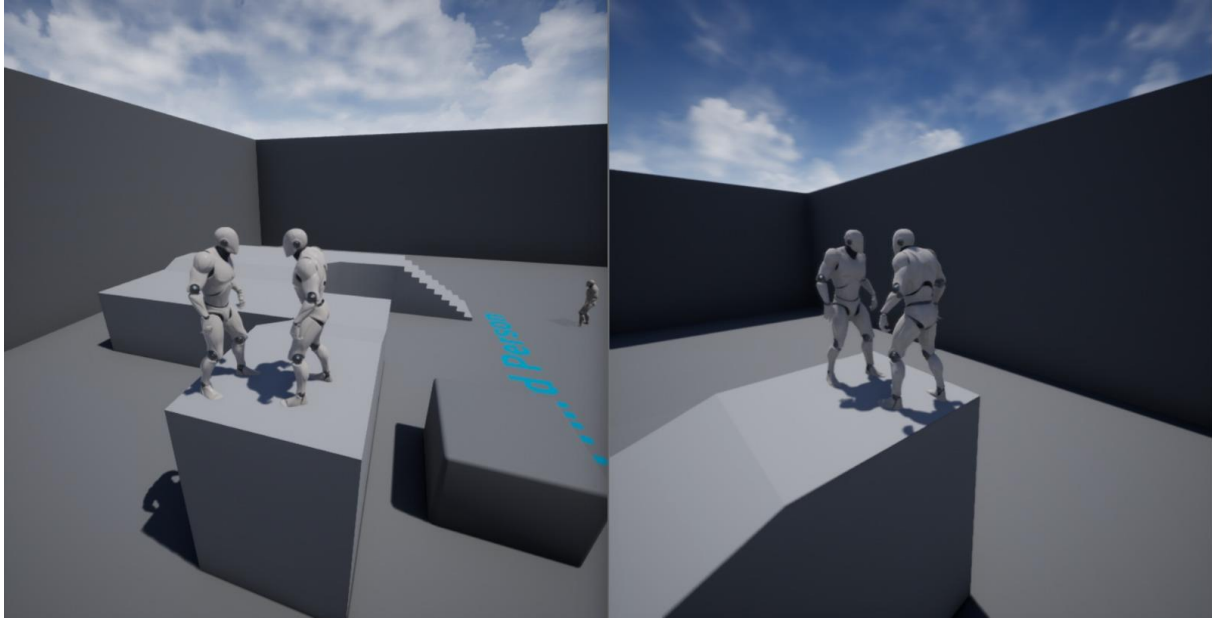
- Stable replication of character movement and firing mechanics confirmed across multiple clients.



3.3 Multiplayer Gameplay Features

During LAN testing, we verified:

- Real-time movement and character control replication
- Replication of shooting mechanics, using projectile components and server-authoritative damage detection
- Visibility of actions between all connected players
- Smooth session join/leave behaviour without server crashes



3.4 Mobile Packaging & Testing

We attempted to build and package mobile versions (APK and IPA) for Android and iOS. The game was tested on emulators and simulators, with promising results. Features like input control, map navigation, and rendering worked well. However, real device testing is still pending, due to hardware access limitations.

Challenges faced:

- Adjusting input controls for touch-based gameplay
- UI scaling for multiple screen resolutions
- Packaging for iOS due to Apple's signing and provisioning complexity

3.5 Pixel Streaming Evaluation

Pixel Streaming allows the game to be rendered on a server and streamed to clients via browsers using WebRTC. We conducted local tests using Unreal Engine's Pixel Streaming setup and observed several critical limitations:

- Each connected client required a separate game instance on the host machine.
- Streaming multiple sessions simultaneously consumed significant memory and CPU.
- High network bandwidth requirements made it unsuitable for users with limited data plans.

- Input latency and inconsistent behaviour on mobile browsers led to a subpar user experience.

Due to these factors, we decided to create a native deployment, and packaging.

4. Test Plan and Expanded Test Cases

Purpose	Result
Connect multiple clients to dedicated server and verify synchronization	✓ Done
Auto-connect on app with a start IP	✓ Done
Test movement and shooting replication	✓ Done
Verify packaging and multiplayer on emulators	⚠ Done on emulator, pending real device
Disconnect and reconnect to server without data loss	✓ Works in LAN test
Stress test multiple simultaneous connections	⌚ Planned

We aim to test multiplayer performance over the internet (with firewall/NAT configuration) and run latency tests on 4G/5G mobile networks once we gain access to mobile hardware.

5. Challenges and Lessons Learned

- Mobile packaging was more difficult than expected due to cross-platform compatibility issues and Unreal Engine mobile limitations.
- Pixel Streaming proved technically viable but inefficient and unsustainable for user and developers. We learned that using pixel streaming is not feasible.
- We learned that planning for platform-specific optimization from the beginning would have saved time later in development.

- Understanding and configuring Unreal networking replication was key to solving sync-related bugs.
- Testing on real hardware is critical; simulator performance doesn't reflect real-world behavior, especially on mobile networks. So we need to deploy to real devices.

6. Conclusion

This report summarizes the Server Module development for our Unreal Engine multiplayer game. We built a reliable dedicated server setup, fully tested multiplayer interactions such as movement and shooting, and confirmed LAN-based synchronization between clients.

Initially, Pixel Streaming was considered to unify browser access across devices. However, after discovering its high hardware and bandwidth requirements, and evaluating modern mobile capabilities, we shifted to a native deployment strategy. This not only improved performance and data efficiency but also offered better long-term support for Android and iOS users.

The final deployment strategy is more challenging due to the need for packaging and testing per platform, but it allows for a smoother, more responsive experience. We continue to refine our mobile builds and look forward to validating the full experience on real devices.

Kinect	Game	Server
Musab KARDEŞ	Bekir Sadık ALTUNKAYA	Mert UĞUR
Beyzanur YILMAZ	Muhammet Talha MEMİŞOĞLU	Ahmet Hakan DEMİR
Ahmet Eren ARSLAN	Mehmet Emin BAYTEKİN	Beyzanur YILMAZ
Ahmet Hakan DEMİR	Ahmet Eren ARSLAN	Musab KARDEŞ
	Mert UĞUR	